Final Project: Cloud-Native AI Application Deployment

Project Overview

Students will design and deploy a fully functional AI-powered application in the cloud using modern cloud services and infrastructure-as-code practices. The project leverages free-tier cloud services and open AI models to create a practical, cost-effective solution.

Project Requirements & Scope

Core Requirements

- Cloud Provider: Yandex Cloud, Google Cloud, or AWS (free tier eligible)
- AI Integration: Google Gemini, Yandex GPT, or any freely available AI model API
- Infrastructure: Terraform for all cloud resource provisioning
- Minimum Services: 4 different cloud services integrated
- Cost Constraint: Must operate within free tier limits

Technical Stack Options

AI Model Choices:

- Google Gemini API (free tier available)
- Yandex GPT (Yandex Cloud)
- Hugging Face Inference API
- OpenAI API (free credits available)

Cloud Architecture Patterns:

- Serverless AI API with Function-as-a-Service
- Containerized AI application with managed Kubernetes
- Static website with AI backend
- Event-driven AI processing pipeline

Project Ideas

1. AI-Powered Content Generator

Architecture:

- Frontend: Static website (S3/Cloud Storage)
- Backend: Cloud Functions/Lambda (AI API integration)
- Database: NoSQL database (DynamoDB/Firestore)
- CDN: Cloud Delivery Network
- AI Service: Text generation via Gemini/YandexGPT

Features:

- Blog post generation
- Social media content creation
- Email writing assistant

 2. Smart Image Analysis Platform
Architecture:
- Storage: Cloud Storage for images
- Processing: Cloud Functions on upload triggers
- AI: Vision model analysis (Google Vision/Yandex Vision)
- Database: Metadata storage
- AI Service: Image recognition and labeling

Features:
- Automatic image tagging
- Content moderation
- Object detection reports

 3. Educational Chatbot Service
Architecture:
- Web Interface: Containerized frontend
- Backend: Microservices architecture
- AI: Conversational AI model
- Database: User sessions and history
- AI Service: Dialogflow or custom chatbot

Features:
- Course material Q&A
- Student assistance
- Learning recommendations

 4. Data Analysis & Visualization Dashboard
Architecture:
- Data Processing: Batch processing functions
- AI: Data analysis and insights generation
- Visualization: Web dashboard
- Storage: Time-series database
- AI Service: Predictive analytics and trend detection

 Implementation Requirements

 Infrastructure as Code

```
project-repo/
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
├── src/
│   ├── application-code
│   └── deployment-scripts
└── docs/
```

```
    ├── architecture.md
    └── setup-guide.md
```

Required Components

1. Networking: VPC, subnets, security groups
2. Compute: Cloud Functions, Containers, or VMs
3. Storage: Object storage and/or databases
4. AI Integration: Working model inference
5. Access Control: IAM roles and permissions

Free Tier Considerations

Yandex Cloud (Recommended)

- Cloud Functions: 1M requests/month
- Object Storage: 1 GB-month
- Managed Databases: Limited hours
- Yandex GPT: Free tier available

Google Cloud

- Cloud Functions: 2M invocations/month
- Cloud Run: 180,000 vCPU-seconds/month
- Firestore: 1 GB storage
- Gemini API: Free quota available

AWS

- Lambda: 1M requests/month
- S3: 5 GB storage
- DynamoDB: 25 GB storage
- API Gateway: 1M API calls/month

Deliverables

1. Terraform Infrastructure Code

- Version-controlled GitHub repository
- Modular, reusable code structure
- Environment-specific configurations
- Documentation for deployment

2. Live Deployed Solution

- Fully functional application
- Accessible via public URL
- All integrated services operational
- AI model responding to requests

3. Technical Documentation

Architecture Documentation:

- System architecture diagram

- Service interaction flows
- Data flow diagrams
- Security considerations

Operational Documentation:
- Deployment instructions
- Cost analysis and optimization
- Monitoring setup
- Troubleshooting guide

 4. Demonstration Script
- 10-minute live demonstration
- Feature showcase
- Architecture walkthrough
- Q&A preparation

 Evaluation Criteria

 Architecture & Design (25%)
- Cloud best practices implementation
- Scalability and reliability design
- Security measures
- Cost optimization strategies

 Implementation & Functionality (35%)
- Successful deployment and operation
- AI integration functionality
- Service interoperability
- Performance and responsiveness

 Infrastructure as Code (20%)
- Terraform code quality and organization
- Modularity and reusability
- State management
- Variable and output design

 Documentation & Demo (20%)
- Documentation completeness and clarity
- Professional demonstration
- Effective communication
- Q&A performance

 Timeline & Milestones

Week 1-2: Project Proposal & Architecture Design
- Technology stack selection
- Architecture diagram completion
- Free-tier verification

Week 3-6: Implementation & Deployment
- Terraform infrastructure setup
- Application development
- AI integration
- Initial deployment

Week 7-8: Testing & Optimization
- Functional testing
- Performance optimization
- Cost analysis
- Documentation completion

Week 9: Final Demonstration
- Live demo preparation
- Documentation submission
- Code repository finalization

Success Tips

1. Start Simple: Begin with basic functionality, then add features
2. Monitor Costs: Use cloud cost calculators and monitor spending
3. Test Early: Deploy frequently to catch issues early
4. Document As You Go: Keep documentation updated during development
5. Use Version Control: Commit code regularly with descriptive messages

Support Resources

- Cloud provider free tier documentation
- Terraform registry and documentation
- AI model API documentation
- Course lab materials and examples
- Instructor office hours

This project demonstrates real-world cloud engineering skills while maintaining cost-effectiveness through free-tier services and open AI technologies.